



COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Unit Convenor)
ptsai@swin.edu.au, EN508d





Week 10



**Multi-Dataset
Operations with
Pig**



MULTI-DATASET OPERATIONS WITH PIG

Main Topics for This Week

How to use grouping to combine data from multiple sources.

Types of join operations in Pig and how to use them.

Concatenating records to produce a single data set.

Splitting a single data set into multiple relations.

Overview of Combining Data Sets

Achievement we made

- In the previous exercises in the labs, we have concentrated on processing single data sets.
- The more valuable insight often results from combining multiple data sets.

Pig offers several techniques for achieving this.

- Using the *GROUP* operator with multiple relations.
- Joining the data as you would in SQL.
- Performing set operations like *CROSS* and *UNION*.

Example Data Sets

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn
B03	Brighton
C01	Camberwell
C02	Clayton

Most examples in this lecture will involve the same two data sets for easy comparisons.

The first is a file containing information about company D's stores.

There are two fields in this relation:

- store_id:chararray (*unique key*)
- name:chararray (*name of the city in which the store is located*)

Example Data Sets

The data set is a file containing the information about company D's salespeople.

This relation contains three fields:

- **person_id**:int (*unique key*)
- **name**:chararray (*salesperson name*)
- **store_id**:chararray (*refers to store*)

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn
B03	Brighton
C01	Camberwell
C02	Clayton

Salespeople

person_id	name	store_id
1	Alice	C01
2	Bill	A01
3	Bruce	B02
4	Charlie	C02
5	Chris	B02
6	Daisy	B01
7	Edward	C01
8	Edwin	A01
9	Toney	C02
10	Natalia	

Grouping Multiple Relations

We previously learned about the *GROUP* operator.

- Groups values in a relation based on the specified field(s).

The *GROUP* operator can also group multiple relations.

- In this case, using the synonymous *COGROUP* operator is preferred.

```
grouped = COGROUP stores BY store_id, salespeople BY store_id;
```

This collects values from both data sets into a new relation.

- As before, the new relation is keyed by a field named *group*.
- This *group* field is associated with one bag of each input.

```
(group, {bag of records}, {bag of records})
```

store_id

records from **stores**

records from **salespeople**

Example of *COGROUP*

```
grunt> grouped = COGROUP stores BY store_id, salespeople BY store_id;
grunt> DUMP grouped;
(A01, {(A01, Ashwood)}, {(2,Bill,A01), (8,Edwin,A01)})
(B01, {(B01, Box Hill)}, {(6,Daisy,B01)})
(B02, {(B02, Balwyn)}, {(5,Chris,B02), (3,Bruce,B02)})
(C01, {(C01, Camberwell)}, {(1,Alice,C01), (7,Edward,C01)})
(C02, {(C02, Clayton)}, {(4,Charlie,C02), (9,Toney,C02)})
(, {}, {(10,Natalia,)})
```

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn
B03	Brighton
C01	Camberwell
C02	Clayton

Salespeople

person_id	name	store_id
1	Alice	C01
2	Bill	A01
3	Bruce	B02
4	Charlie	C02
5	Chris	B02
6	Daisy	B01
7	Edward	C01
8	Edwin	A01
9	Toney	C02
10	Natalia	

Join Data Sets in Pig

The *COGROUP* creates a nested data structure.

Pig Latin's *JOIN* operator creates a flat data structure.

- Similar to joins in a relational database.

A *JOIN* is similar to doing a *COGROUP* followed by a *FLATTEN*.

- But, they handle null values differently.

Key Fields

Like *COGROUP*, joins rely on a field shared by each relation.

```
joined = JOIN stores BY store_id, salespeople BY store_id;
```

Joins can also use multiple fields as the key.

```
joined = JOIN customers BY (name, phone_number),  
           accounts BY (name, phone_number);
```


Inner Joins

The default *JOIN* in Pig Latin is an inner join:

```
joined = JOIN stores BY store_id, salespeople BY store_id;
```

An inner join outputs records only when the key is found in **all** inputs.

- In the above example, stores that have at least one salesperson.

You can do an inner join on multiple relations in a single statement.

- But you must use the same key to join them.

Inner Join Example

```
grunt> joined = JOIN stores BY store_id, salespeople BY store_id;  
grunt> DUMP joined;  
(A01, Ashwood, 2, Bill, A01)  
(A01, Ashwood, 8, Edwin, A01)  
(B01, Box Hill, 6, Daisy, B01)  
(B02, Balwyn, 5, Chris, B02)  
(B02, Brighton, 3, Bruce, B02)  
(C01, Camberwell, 1, Alice, C01)  
(C01, Camberwell, 7, Edward, C01)  
(C02, Clayton, 4, Charlie, C02)  
(C02, Clayton, 9, Toney, C02)
```

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn
B03	Brighton
C01	Camberwell
C02	Clayton

Salespeople

person_id	name	store_id
1	Alice	C01
2	Bill	A01
3	Bruce	B02
4	Charlie	C02
5	Chris	B02
6	Daisy	B01
7	Edward	C01
8	Edwin	A01
9	Toney	C02
10	Natalia	

Eliminating Duplicate Fields

```
grunt> grouped = COGROUP stores BY store_id, salespeople BY store_id;
grunt> DUMP grouped;
(A01, {(A01, Ashwood)}, {(2, Bill, A01), (8, Edwin, A01)})
(B01, {(B01, Box Hill)}, {(6, Daisy, B01)})
(B02, {(B02, Balwyn)}, {(5, Chris, B02), (3, Bruce, B02)})
(C01, {(C01, Camberwell)}, {(1, Alice, C01), (7, Edward, C01)})
(C02, {(C02, Clayton)}, {(4, Charlie, C02), (9, Toney, C02)})
(, {}, {(10, Natalia, )})
```

```
grunt> joined = JOIN stores BY store_id, salespeople BY store_id;
grunt> DUMP joined;
(A01, Ashwood, 2, Bill, A01)
(A01, Ashwood, 8, Edwin, A01)
(B01, Box Hill, 6, Daisy, B01)
(B02, Balwyn, 5, Chris, B02)
(B02, Brighton, 3, Bruce, B02)
(C01, Camberwell, 1, Alice, C01)
(C01, Camberwell, 7, Edward, C01)
(C02, Clayton, 4, Charlie, C02)
(C02, Clayton, 9, Toney, C02)
```

- After performing *COGROUP* or *JOIN*, the new relation still contains duplicate fields.

Eliminating Duplicate Fields

- We can use *FOREACH...GENERATE* to retain just the fields we need.
- However, it is now slightly more complex to reference fields.
- We must fully-qualify any fields with names that are not unique.

```
grunt> DESCRIBE joined;  
joined: {stores::store_id:chararray, stores::name:chararray,  
salespeople::person_id:int, salespeople::name:chararray,  
salespeople::store_id:chararray}
```

```
grunt> cleaned = FOREACH joined GENERATE stores::store_id,  
stores::name, salespeople::person_id, salespeople::name;
```

```
grunt> DUMP cleaned;  
(A01, Ashwood, 2, Bill)  
(A01, Ashwood, 8, Edwin)  
(B01, Box Hill, 6, Daisy)  
(B02, Balwyn, 5, Chris)  
(B02, Brighton, 3, Bruce)  
(C01, Camberwell, 1, Alice)  
(C01, Camberwell, 7, Edward)  
(C02, Clayton, 4, Charlie)  
(C02, Clayton, 9, Toney)
```


Outer Joins

Pig Latin allows you to specify the type of join following the field name.

- Inner joins not specify a join type as it is being used as the default join method.

```
joined = JOIN relation1 BY field [LEFT|RIGHT|FULL] OUTER, relation2 BY field;
```

An outer join does not require the key to be found in the input sources.

Outer joins require Pig to know the schema for at least one relation.

- Which relation requires schema depends on the join type.
- Full outer joints require schema for both relations.

Left Outer Join Example

- Result contains *all* records from the relation specified on the *left*, but only *matching* records from the one specified on the *right*.

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn
B03	Brighton
C01	Camberwell
C02	Clayton

Salespeople

person_id	name	store_id
1	Alice	C01
2	Bill	A01
3	Bruce	B02
4	Charlie	C02
5	Chris	B02
6	Daisy	B01
7	Edward	C01
8	Edwin	A01
9	Toney	C02
10	Natalia	

```
grunt> joined = JOIN stores BY store_id LEFT OUTER, salespeople BY  
store_id;
```

```
grunt> DUMP joined;
```

```
(A01, Ashwood, 2, Bill, A01)
```

```
(A01, Ashwood, 8, Edwin, A01)
```

```
(B01, Box Hill, 6, Daisy, B01)
```

```
(B02, Balwyn, 5, Chris, B02)
```

```
(B02, Brighton, 3, Bruce, B02)
```

```
(B03, Brighton,,)
```

```
(C01, Camberwell, 1, Alice, C01)
```

```
(C01, Camberwell, 7, Edward, C01)
```

```
(C02, Clayton, 4, Charlie, C02)
```

```
(C02, Clayton, 9, Toney, C02)
```

Right Outer Join Example

- Result contains *all* records from the relation specified on the *right*, but only *matching* records from the one specified on the *left*.

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn
B03	Brighton
C01	Camberwell
C02	Clayton

Salespeople

person_id	name	store_id
1	Alice	C01
2	Bill	A01
3	Bruce	B02
4	Charlie	C02
5	Chris	B02
6	Daisy	B01
7	Edward	C01
8	Edwin	A01
9	Toney	C02
10	Natalia	

```
grunt> joined = JOIN stores BY store_id RIGHT OUTER, salespeople BY store_id;
```

```
grunt> DUMP joined;
```

```
(A01, Ashwood, 2, Bill, A01)
```

```
(A01, Ashwood, 8, Edwin, A01)
```

```
(B01, Box Hill, 6, Daisy, B01)
```

```
(B02, Balwyn, 5, Chris, B02)
```

```
(B02, Brighton, 3, Bruce, B02)
```

```
(C01, Camberwell, 1, Alice, C01)
```

```
(C01, Camberwell, 7, Edward, C01)
```

```
(C02, Clayton, 4, Charlie, C02)
```

```
(C02, Clayton, 9, Toney, C02)
```

```
(,, 10, Natalia,)
```

Full Outer Join Example

- Result contains *all* records where there is a match in *either* relation.

```
grunt> joined = JOIN stores BY store_id FULL OUTER, salespeople BY store_id;
```

```
grunt> DUMP joined;
```

```
(A01, Ashwood, 2, Bill, A01)
```

```
(A01, Ashwood, 8, Edwin, A01)
```

```
(B01, Box Hill, 6, Daisy, B01)
```

```
(B02, Balwyn, 5, Chris, B02)
```

```
(B02, Brighton, 3, Bruce, B02)
```

```
(B03, Brighton,,)
```

```
(C01, Camberwell, 1, Alice, C01)
```

```
(C01, Camberwell, 7, Edward, C01)
```

```
(C02, Clayton, 4, Charlie, C02)
```

```
(C02, Clayton, 9, Toney, C02)
```

```
(,, 10, Natalia,)
```

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn
B03	Brighton
C01	Camberwell
C02	Clayton

Salespeople

person_id	name	store_id
1	Alice	C01
2	Bill	A01
3	Bruce	B02
4	Charlie	C02
5	Chris	B02
6	Daisy	B01
7	Edward	C01
8	Edwin	A01
9	Toney	C02
10	Natalia	

Set Operation – Crossing Data Sets

JOIN finds records in one relation that match records in another.

Pig's *CROSS* operator creates the cross product of both relations.

- Combines all records in both tables regardless of matching.
- In other words, all possible combinations of records.

`crossed = CROSS stores, salespeople;`

Careful: This can generate huge amounts of data!

Cross Product Example

- Generates every possible combination of records in the *stores* and *salespeople* relations.

```
grunt> crossed = CROSS stores, salespeople;
```

```
grunt> DUMP crossed;
```

```
(A01, Ashwood, 1, Alice, C01)
```

```
(A01, Ashwood, 2, Bill, A01)
```

```
(A01, Ashwood, 3, Bruce, B02)
```

```
(A01, Ashwood, 10, Natalia,)
```

```
(B01, Box Hill, 1, Alice, C01)
```

```
(B01, Box Hill, 2, Bill, A01)
```

```
(B01, Box Hill, 3, Bruce, B02)
```

```
(B01, Box Hill, 10, Natalia,)
```

```
(B02, Balwyn, 1, Alice, C01)
```

```
(B02, Balwyn, 2, Bill, A01)
```

```
(B02, Balwyn, 3, Bruce, B02)
```

```
(B02, Balwyn, 10, Natalia,)
```

Stores

store_id	name
A01	Ashwood
B01	Box Hill
B02	Balwyn

Salespeople

person_id	name	store_id
1	Alice	C01
2	Bill	A01
3	Bruce	B02
10	Natalia	

Concatenating Data Sets

We have explored several techniques for combining data sets.

- They have had one thing in common: they combine *horizontally*.

The *UNION* operator combines records *vertically*.

- It add data from input relations into a new single relation.
- Pig does not require these inputs to have the same schema.
- It does not eliminate duplicate records nor preserve order.

This is helpful incorporating new data into your processing.

```
both = UNION relation_1, relation_2;
```

Union Example

- Concatenates all records from *Saturday* and *Sunday*.

```
grunt> both = UNION Saturday, Sunday;  
grunt> DUMP both;  
(Lamb,21.95)  
(Chips,5.99)  
(Baked Potato,7.99)  
(Steamed Fish,23.49)  
(Roasted Chicken,18.99)  
(BBQ T-Bone,20.99)  
(Specialty Pizza,16.99)  
(Perl Bubble Tea,7.99)
```

Saturday

Dish	Price
Lamb	21.95
Chips	5.99
Baked Potato	7.99

Sunday

Dish	Price
Steamed Fish	23.49
Roasted Chicken	18.99
BBQ T-Bone	20.99
Specialty Pizza	16.99
Perl Bubble Tea	7.99

Splitting Data Sets

You have learned several ways to combine data sets into a single relation.

Sometimes you need to split a data set into multiple relations.

- Server logs by date range.
- Customer lists by region.
- Product lists by vendor.

Pig Latin supports this with the *SPLIT* operator.

```
SPLIT relation INTO relationA IF expression1,  
                      relationB IF expression2,  
                      relationC IF expression3...;
```

- Expressions need not be mutually exclusive.

SPLIT Example

- Split customers into groups for the reward program based on lifetime value.

```
grunt> SPLIT customers INTO golden_program IF Value >= 25000,  
silver_program IF Value >= 10000 AND Value < 25000;
```

```
grunt> DUMP golden_program;  
(Bill,32350)  
(Gina,28760)  
(Ivan,44000)  
(Kate,35000)
```

```
grunt> DUMP silver_program;  
(Chris,18200)  
(Dan,21350)  
(John,23120)  
(Mike,24200)
```

Customers

Name	Value
Ameda	2840
Bill	32350
Chris	18200
Dan	21350
Eve	8200
Felix	9500
Gina	28760
Henry	8800
Ivan	44000
John	23120
Kate	35000
Laura	6900
Mike	24200

Essential Points

You can use *COGROUP* to group multiple relations.

- This creates a nested data structure.

Pig supports common SQL join types.

- Inner, left outer, right outer, and full outer joins.
- You may need to fully-qualify field names when using joined data.

Pig's *CROSS* operator creates every possible combination of input data.

- This can create huge amounts of data – use it carefully!

You can use a *UNION* to concatenate data sets.

In addition to combining datasets, Pig supports splitting them too.



EXTENDING PIG

The Need for Parameters

Some processing is very repetitive.

- For example, creating sales reports.

```
allsales = LOAD 'sales' AS (name:chararray, price:int);  
bigsales = FILTER allsales BY price > 999;
```

```
bigsales_alice = FILTER bigsales BY name == 'Alice';  
STORE bigsales_alice INTO 'Alice';
```

Making the Script More Flexible with Parameters

Instead of hardcoding values, Pig allows you to use parameters.

- These are replaced with specified values at runtime.

```
allsales = LOAD '$INPUT' AS (name:chararray, price:int);  
bigsales = FILTER allsales BY price > $MINPRICE;
```

```
bigsales_alice = FILTER bigsales BY name == '$NAME';  
STORE bigsales_alice INTO '$NAME';
```

- Then specify the values on the command line.

```
$ pig -p INPUT='sales' -p MINPRICE=999 -p NAME='Alice' reporter.pig
```

Two Tricks for Specifying Parameter Values

You can also specify parameter values in a text file.

- An alternative to typing each one on the command line.

```
INPUT='sales'  
MINPRICE=999  
# comments look like this  
NAME='Alice'
```

- Use `-m filename` option to tell Pig which file contains the values.

Parameter values can be defined with the output of a shell command.

- For example, to set *MONTH* to the current month:

```
MONTH=`date +%m` # returns 03 for March, 10 for October
```




PIG TROUBLESHOOTING AND OPTIMISATION

Following Topics

How to control the information that Pig and Hadoop write to log files?

How Hadoop's Web UI can help you troubleshoot failed jobs?

How to use *SAMPLE* and *ILLUSTRATE* to test and debug Pig jobs?

How Pig creates MapReduce jobs from your Pig Latin code?

How several simple changes to your Pig Latin code can make it run faster?

Which resources are especially helpful for troubleshooting Pig errors?

Troubleshooting Overview

We have now covered how to use Pig for data analysis.

- **Unfortunately, sometimes your code may not work as you expected.**
- **It is important to remember that Pig and Hadoop are intertwined.**

Here we will cover some techniques for isolating and resolving problems.

- **We will start with a few options to the Pig command.**

Helping Yourself

We will discuss some options for the Pig command in this chapter.

- You can view all of them by using the `-h` (help) option.
- Keep in mind that many options are advanced or rarely used.

An useful option is `-c` (check), which validates the syntax of your code.

```
$ pig -c myscript.pig  
myscript.pig syntax OK
```

The `-dryrun` option is very helpful if you use parameters or macros.

```
$ pig -p INPUT=demodata -dryrun myscript.pig
```

- Create a *myscript.pig.substituted* file in current directory.

```
[training@192-168-1-109 week10]$ pig -x local -dryrun union_test.pig  
2020-09-28 01:36:14,187 INFO org.apache.pig.Main: Apache Pig version 0.10.0-cdh4  
.2.1 (rexported) compiled Apr 22 2013, 12:04:54  
2020-09-28 01:36:14,187 INFO org.apache.pig.Main: Logging error messages to: /ho  
me/training/week10/pig_1601271374183.log  
2020-09-28 01:36:15,453 INFO org.apache.pig.Main: Dry run completed. Substituted  
pig script is at union_test.pig.substituted  
[training@192-168-1-109 week10]$
```

Getting Help from Others

Sometimes you may need help from others.

- Mailing lists or newsgroups.
- Forums and bulletin board sites.
- Support services.

You will probably need to provide the version of Pig and Hadoop you are using.

```
[training@192-168-1-109 week10]$ pig -version
Apache Pig version 0.10.0-cdh4.2.1 (rexported)
compiled Apr 22 2013, 12:04:54
[training@192-168-1-109 week10]$ hadoop version
Hadoop 2.0.0-cdh4.2.1
Subversion file:///data/1/jenkins/workspace/generic-package-rhel64-6-0/topdir/BU
ILD/hadoop-2.0.0-cdh4.2.1/src/hadoop-common-project/hadoop-common -r 144bd548d48
1c2774fab2bec2ac2645d190f705b
Compiled by jenkins on Mon Apr 22 10:26:03 PDT 2013
From source with checksum aef88defdddfb22327a107fbd7063395
[training@192-168-1-109 week10]$ █
```

Logging: Customising Log Messages

You may wish to change how much information is logged.

- A recent change in Hadoop can cause lots of warnings when using Pig.

Pig and Hadoop use the Log4J library, which is easily customised.

Edit the */etc/pig/conf/log4j.properties* file to include:

```
log4j.logger.org.apache.pig=ERROR  
log4j.logger.org.apache.hadoop.config.Configuration=ERROR
```

Edit the */etc/pig/conf/pig.properties* file to set this property:

```
log4jconf=/etc/pig/conf/log4j.properties
```

Customising Log Messages on a Per-Job Basis

Often you just want to temporarily change the log level.

- Especially while trying to troubleshoot a problem with your script.

You can specify a *Log4J* properties file to use when you invoke Pig.

- This overrides the default Log4J configuration.

Create a customlog.properties file to include:

```
log4j.logger.org.apache.pig=DEBUG
```

Specify this file via the `-log4jconf` argument to Pig

```
$ pig -log4jconf customlog.properties
```

Controlling Client-Side Log Files

When a job fails, Pig may produce a log file to explain why.

- These are typically produced in your current directory.

To use a different location, use the `-l (log)` option when starting Pig.

```
$pig -l /tmp
```

Or set it permanently by editing */etc/pig/conf/pig.properties*

- Specify a different directory using the *log.file* property.

```
log.file=/tmp
```

Using Hadoop's Web UI

Each Hadoop daemon has a corresponding Web application.

- This allows us to easily see cluster and job status with a browser.
- In pseudo-distributed mode, the hostname is *localhost*.

	Daemon Name	Address
HDFS	NameNode	http://hostname:50070/
	DataNode	http://hostname:50075/
MR1	JobTracker	http://hostname:50030/
	TaskTracker	http://hostname:50060/
MR2	ResourceManager	http://hostname:8088/
	NodeManager	http://hostname:8042/

The JobTracker Web UI

- The JobTracker offers the most useful of Hadoop's Web Uis.
 - It displays MapReduce status information for the Hadoop cluster.

0.0.0.0 Hadoop Map/Reduce Administration

State: RUNNING
Started: Sun Sep 27 07:59:54 EDT 2020
Version: 2.0.0-mr1-cdh4.2.1, Unknown
Compiled: Mon Apr 22 11:06:35 PDT 2013 by jenkins from Unknown
Identifier: 202009270759

Cluster Summary (Heap Size is 60 MB/193.38 MB)

Running Map Tasks	Running Reduce Tasks	Total Submissions	Nodes	Occupied Map Slots	Occupied Reduce Slots	Reserved Map Slots	Reserved Reduce Slots	Map Task Capacity	Reduce Task Capacity	Avg. Tasks/Node	Blacklisted Nodes	Excluded Nodes
0	0	43	1	0	0	0	0	2	2	4.00	0	0

Scheduling Information

Queue Name	State	Scheduling Information
default	running	N/A

Filter (Jobid, Priority, User, Name)
Example: 'user:smith 3200' will filter by 'smith' only in the user field and '3200' in all fields

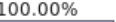
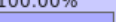
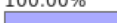
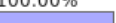
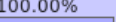
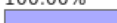
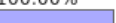
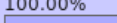
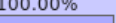
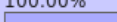
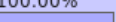
Running Jobs

Completed Jobs

The JobTracker Web UI

- The JobTracker Web UI also shows historical information
 - You can click one of the links to see details for a particular job.

Completed Jobs

Jobid	Priority	User	Name	Map % Complete	Map Total	Maps Completed	Reduce % Complete	Reduce Total	Reduces Completed	Job Sched Inform
job_202009270759_0002	NORMAL	training	customers.jar	100.00% 	4	4	100.00% 	0	0	NA
job_202009270759_0003	NORMAL	training	products.jar	100.00% 	4	4	100.00% 	0	0	NA
job_202009270759_0004	NORMAL	training	orders.jar	100.00% 	4	4	100.00% 	0	0	NA
job_202009270759_0005	NORMAL	training	order_details.jar	100.00% 	4	4	100.00% 	0	0	NA
job_202009270759_0006	NORMAL	training	PigLatin:first_etl.pig	100.00% 	1	1	100.00% 	0	0	NA
job_202009270759_0007	NORMAL	training	PigLatin:first_etl.pig	100.00% 	1	1	100.00% 	0	0	NA
job_202009270759_0009	NORMAL	training	PigLatin:second_etl.pig	100.00% 	1	1	100.00% 	1	1	NA
job_202009270759_0010	NORMAL	training	PigLatin:low_cost_sites.pig	100.00% 	1	1	100.00% 	1	1	NA

The JobTracker Web UI

- The job detail page can help you troubleshoot a problem.

Hadoop job_201303151454_0006 on localhost

User: training

Job Name: PigLatin:x.pig

Job File: hdfs://0.0.0.0:8020/var/lib/hadoop-hdfs/cache/mapred/mapred/staging/training/.staging/job_201303151454_0006/job.xml

Submit Host: localhost.localdomain

Submit Host Address: 127.0.0.1

Job-ACLs: All users are allowed

Job Setup: Successful

Status: Failed

Failure Info: NA

Started at: Wed Mar 20 11:38:29 EDT 2013

Failed at: Wed Mar 20 11:39:20 EDT 2013

Failed in: 50sec

Job Cleanup: Successful

Black-listed TaskTrackers: 1

Kind	% Complete	Num Tasks	Pending	Running	Complete	Killed	Failed/Killed Task Attempts
map	100.00%	1	0	0	1	0	0 / 0
reduce	100.00%	1	0	0	0	1	4 / 0

	Counter	Map	Reduce	Total
	FILE: Number of bytes read	0	0	0
	FILE: Number of bytes written	351,264	0	351,264
	FILE: Number of read operations	0	0	0

Naming Your Job

Hadoop clusters are typically shared resources.

- There might be dozens or hundreds of others using it.
- As a result, sometimes it is hard to find your job in the Web UI.

We recommend setting a name in your scripts to help you identify your jobs.

- Set the *job.name* property, either in Grunt or your script.

```
grunt> set job.name 'first_etl.pig';
```

Jobid	Priority	User	Name
job_202009270759_0006	NORMAL	training	PigLatin:first_etl.pig
job_202009270759_0007	NORMAL	training	PigLatin:first_etl.pig
job_202009270759_0009	NORMAL	training	PigLatin:second_etl.pig

Killing a Job

A job that processes a lot of data can take hours to complete.

- Sometimes you spot an error in your code just after submitting a job.
- Rather than wait for the job to complete, you can kill it.

First, find the Job ID on the front page of the JobTracker Web UI.

Running Jobs

Jobid	Priority	User	Name	Map % Complete
job_201303151454_0028	NORMAL	training	PigLatin:Q2 2013 Sales Reporter	0.00%

Then, use the *kill* command in Pig along with that Job ID.

```
grunt> kill job_201303151454_0028;
```

Using SAMPLE to Create a Smaller Data Set

Your code might process terabytes of data in production.

- However, it is convenient to test with smaller amounts during development.

Use *SAMPLE* to choose a random set of records from a data set.

This example selects about 5% of records from *bigdata*.

- Stores them in a new directory called *mysample*.

```
everything = LOAD 'bigdata';  
subset = SAMPLE everything 0.05;  
STORE subset INTO 'mysample';
```

Intelligent Sampling with *ILLUSTRATE*

Sometimes a random sample may lack data needed for testing.

- For example, matching records in two data set for a *JOIN* operation.

Pig's *ILLUSTRATE* keyword can do more intelligent sampling.

- Pig will examine the code to determine what data is needed.
- It picks a few records that properly exercise the code.

You should specify a schema when using *ILLUSTRATE*.

- Pig will generate records when yours are not suffice.

Using ILLUSTRATE Helps You to Understand Data Flow

Like *DUMP* and *DESCRIBE*, *ILLUSTRATE* aids in debugging.

- The syntax is the same for all three.

```
grunt> allsales = LOAD 'sales' AS (name:chararray, price:int);
grunt> bigsales = FILTER allsales BY price > 999;
grunt> ILLUSTRATE bigsales;
(Bob,3625)
```

allsales	name:chararray	price:int
	Bob	3625
	Bob	998

bigsales	name:chararray	price:int
	Bob	3625

General Debugging Strategies

Use *DUMP*, *DESCRIBE*, and *ILLUSTRATE* often.

- The data might not be what you think it is.

Look at a sample of the data.

- Verify that it matches the fields in your *Load* specification.

Other helpful steps for tracking down a problem.

- Use *-dryrun* to see the script after parameters and macros are processed.
- Test external scripts (STREAM) by passing some data from a local file.
- Look at the logs, especially task logs available from the Web UI.

Performance Overview

We have discussed several techniques for finding errors in Pig Latin code.

- Once you get your code working, you'll often want it to work *faster*.

Performance tuning is a broad and complex subject.

- Requires a deep understanding of Pig, Hadoop, Java, and Linux.
- Typically the domain of engineers and system administrators.

Most of these topics are beyond the scope of this course.

- We'll cover the basics and offer several performance improvement tips.
- See *Programming Pig* (chapters 7 and 8) for detailed coverage.

Understanding the Execution Plan: How Pig Latin Becomes a MapReduce Job?

Pig Latin code ultimately runs as MapReduce jobs on the Hadoop cluster.

However, Pig does not translate your code into Java MapReduce.

- Much like relational databases don't translate SQL to C language code.
- Like a database, Pig interprets the Pig Latin to develop execution plans.
- Pig's execution engine uses these to submit MapReduce jobs to Hadoop.

The *EXPLAIN* keyword details Pig's three execution plans.

- Logical
- Physical
- MapReduce

Seeing an example job will help us better understanding EXPLAIN's output.

Description of Our Example Code and Data Brands

- Our goal is to produce a list of pre-brand sales.

```
grunt> brands = LOAD 'Brands' AS (brand_id:chararray, name:chararray);
grunt> sales = LOAD 'Sales' AS (brand_id:chararray, volume:int);
grunt> groups = GROUP sales BY brand_id;
grunt> totals = FOREACH groups GENERATE group, SUM(sales.volume) AS amount;
grunt> joined = JOIN totals BY group, brands BY brand_id;
grunt> result = FOREACH joined GENERATE name, amount;
grunt> DUMP result;
(Volkswagen,14632)
(Audi,4748)
(Mercedes-Benz,5389)
(Toyota,1677)
```

brand_id	name
A	Volkswagen
B	Audi
C	BMW
D	Mercedes-Benz
E	Toyota
F	Mazda

Sales

brand_id	Volume
A	3192
D	2590
A	4208
B	3651
A	7232
B	1097
E	1677
D	2799

Using the *EXPLAIN* Keyword

- Using *EXPLAIN* rather than *DUMP* will show the execution plans.
 - Logical Plan.
 - Physical Plan.
 - MapReduce Plan.

```
#-----
# New Logical Plan:
#-----
result: (Name: LOStore Schema: brands::name#100:chararray,totals::amount#121:long)
|
|---result: (Name: LOForEach Schema: brands::name#100:chararray,totals::amount#121:long)
|   |   |
|   |   | (Name: LOGenerate[false,false] Schema: brands::name#100:chararray,totals::amount#121:long)ColumnPrune:InputUids=[100,
|   |   | 121]ColumnPrune:OutputUids=[100, 121]
|   |   |
|   |   | brands::name:(Name: Project Type: chararray Uid: 100 Input: 0 Column: (*))
|   |   |
|   |   | totals::amount:(Name: Project Type: long Uid: 121 Input: 1 Column: (*))
|   |   |
#-----
# Physical Plan:
#-----
result: Store(fakefile:org.apache.pig.builtin.PigStorage) - scope-115
|
|---result: New For Each(false,false)[bag] - scope-114
|   |   |
|   |   | Project[chararray][3] - scope-110
|   |   |
#-----
# Map Reduce Plan
#-----
MapReduce node scope-116
Map Plan
groups: Local Rearrange[tuple]{chararray}(false) - scope-135
|   |
|   | Project[chararray][0] - scope-136
|   |
|---totals: New For Each(false,false)[bag] - scope-123
|   |   |
|   |   | Project[chararray][0] - scope-124
```

Pig's Runtime Optimisations

Pig does not necessarily run your statements exactly as you wrote them.

It may remove operations for efficiency.

```
sales = LOAD 'sales' AS (store_id:chararray, price:int);  
unused = FILTER sales BY price > 789;  
DUMP sales;
```

It may also rearrange operations for efficiency.

```
grouped = GROUP sales BY store_id;  
totals = FOREACH grouped GENERATE group, SUM(sales.price);  
joined = JOIN totals BY group, stores BY store_id;  
only_a = FILTER joined BY store_id == 'A';  
DUMP only_a;
```


Optimisations You Can Make in Your Pig Latin Code

Pig's optimiser does what it can to improve performance.

- But you know your own code and data better than it does.
- A few small changes in your code can allow additional optimisations.

In the next few slides, we will rewrite this Pig code for performance.

```
stores = LOAD 'stores' AS (store_id, name, postcode, phone);
sales = LOAD 'sales' AS (store_id, price);
joined = JOIN sales BY store_id, stores BY store_id;
DUMP joined;
groups = GROUP joined BY sales::store_id;
totals = FOREACH groups GENERATE FLATTEN(joined.stores::name) AS name,
    SUM(joined.sales::price) AS amount;
unique = DISTINCT totals;
region = FILTER unique BY name == 'Hawthorn' OR name == 'Pakenham';
stored = ORDER region BY amount DESC;
topone = LIMIT sorted 1;
STORE topone INTO 'topstore';
```

Don't Produce Output You Don't Really Need

In this case, we forgot to remove the *DUMP* statement.

- Sometimes happens when moving from development to production.
- And it might go unnoticed if you're not watching the terminal.

```
stores = LOAD 'stores' AS (store_id, name, postcode, phone);
sales = LOAD 'sales' AS (store_id, price);
joined = JOIN sales BY store_id, stores BY store_id;
--DUMP joined;
groups = GROUP joined BY sales::store_id;
totals = FOREACH groups GENERATE FLATTEN(joined.stores::name) AS name,
        SUM(joined.sales::price) AS amount;
unique = DISTINCT totals;
region = FILTER unique BY name == 'Hawthorn' OR name == 'Pakenham';
stored = ORDER region BY amount DESC;
topone = LIMIT sorted 1;
STORE topone INTO 'topstore';
```

Specify Schema Whenever Possible

Specifying schema when loading data eliminates the need for Pig to guess.

- It may choose a 'bigger' type than you need. (e.g., *long* instead of *int*)

The postcode and phone fields in the stores data set were also never used.

- Eliminating them in our schema ensures they'll be omitted in *joined*.

```
stores = LOAD 'stores' AS (store_id:chararray, name:chararray);
sales = LOAD 'sales' AS (store_id:chararray, price:int);
joined = JOIN sales BY store_id, stores BY store_id;
--DUMP joined;
groups = GROUP joined BY sales::store_id;
totals = FOREACH groups GENERATE FLATTEN(joined.stores::name) AS name,
    SUM(joined.sales::price) AS amount;
unique = DISTINCT totals;
region = FILTER unique BY name == 'Hawthorn' OR name == 'Pakenham';
stored = ORDER region BY amount DESC;
topone = LIMIT sorted 1;
STORE topone INTO 'topstore';
```

Filter Unwanted Data as Early as Possible.

We previously did our *JOIN* before our *FILTER*.

- This produced lots of data we ultimately discarded.
- Moving the *FILTER* operation up makes our script more efficient.
- Caveat: We now have to filter by store ID rather than store name.

```
stores = LOAD 'stores' AS (store_id:chararray, name:chararray);
sales = LOAD 'sales' AS (store_id:chararray, price:int);
regsales = FILTER sales BY store_id == 'A' OR store_id == 'E';
joined = JOIN stores BY store_id, regsales BY store_id;
--DUMP joined;
groups = GROUP joined BY sales::store_id;
totals = FOREACH groups GENERATE FLATTEN(joined.stores::name) AS name,
        SUM(joined.sales::price) AS amount;
unique = DISTINCT totals;
stored = ORDER region BY amount DESC;
topone = LIMIT sorted 1;
STORE topone INTO 'topstore';
```

Consider Adjusting the Parallelisation

Hadoop clusters scale by processing data in parallel.

- Newer Pig releases choose the number of reducers based on input size.
- However, it is often beneficial to set a value explicitly in your script.
- Your system administrator can help you determine the best value.

```
set default_parallel 5;
```

```
stores = LOAD 'stores' AS (store_id:chararray, name:chararray);  
sales = LOAD 'sales' AS (store_id:chararray, price:int);  
regsales = FILTER sales BY store_id == 'A' OR store_id == 'E';  
joined = JOIN stores BY store_id, regsales BY store_id;  
-- DUMP joined;  
groups = GROUP joined BY sales::store_id;  
totals = FOREACH groups GENERATE FLATTEN(joined.stores::name) AS name,  
      SUM(joined.sales::price) AS amount;  
unique = DISTINCT totals;  
stored = ORDER region BY amount DESC;  
topone = LIMIT sorted 1;  
STORE topone INTO 'topstore';
```

Specify the Smaller Data Set First in a Join

We can optimise joins by specifying the larger data set last.

- Pig will 'stream' the larger data set instead of reading it into memory.
- In our case, we have far more records in *sales* than in *stores*.
- Changing the order in the *JOIN* statement can boost performance.

```
set default_parallel 5;
stores = LOAD 'stores' AS (store_id:chararray, name:chararray);
sales = LOAD 'sales' AS (store_id:chararray, price:int);
regsales = FILTER sales BY store_id == 'A' OR store_id == 'E';
joined = JOIN stores BY store_id, regsales BY store_id;
-- DUMP joined;
groups = GROUP joined BY sales::store_id;
totals = FOREACH groups GENERATE FLATTEN(joined.stores::name) AS name,
    SUM(joined.sales::price) AS amount;
unique = DISTINCT totals;
stored = ORDER region BY amount DESC;
topone = LIMIT sorted 1;
STORE topone INTO 'topstore';
```

Try Using Compression on Intermediate Data

Pig scripts often yield jobs with both a Map and a Reduce phase.

- Remember that Mapper output becomes Reducer's input.
- Compressing this intermediate data is easy and can boost performance.
- Your system administrator may need to install a compression library.

```
set mapred.compress.map.output true;  
set mapred.map.output.compression.codec  
    org.apache.hadoop.io.compress.SnappyCodec;  
set default_parallel 5;  
stores = LOAD 'stores' AS (store_id:chararray, name:chararray);  
sales = LOAD 'sales' AS (store_id:chararray, price:int);  
regsales = FILTER sales BY store_id == 'A' OR store_id == 'E';  
joined = JOIN stores BY store_id, regsales BY store_id;  
-- DUMP joined;  
groups = GROUP joined BY sales::store_id;  
totals = FOREACH groups GENERATE FLATTEN(joined.stores::name) AS name,  
    SUM(joined.sales::price) AS amount;  
... (other lines unchanged, but removed for brevity) ...
```

A Few More Tips for Improving Performance

Main theme: Eliminate unnecessary data as early as possible.

- Use *FOREACH ... GENERATE* to select just those fields you need.
- Use *ORDER BY* and *LIMIT* when you only need a few records.
- Use *DISTINCT* when you don't need duplicated records.

Dropping records with *NULL* keys before a join can boost performance.

- *These records will be eliminated in the final output anyway.*
- *But Pig doesn't discard them until after the join.*
- Use *FILTER* to remove records with null keys before the join.

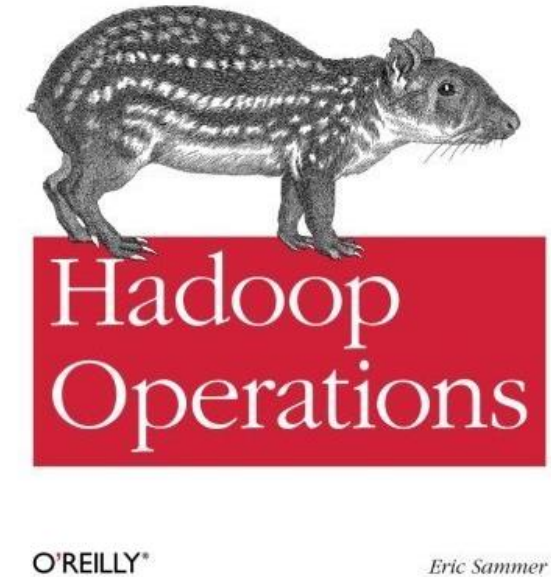
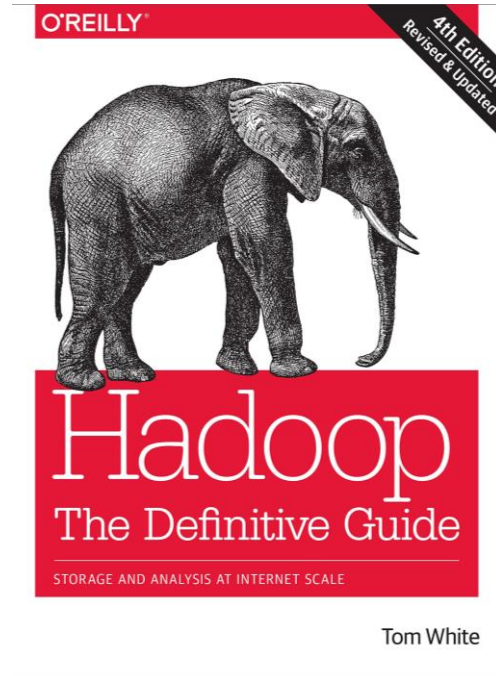
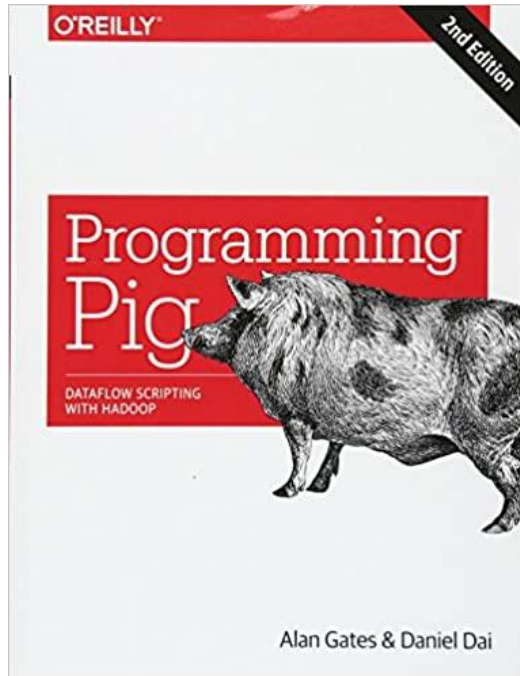
```
stores = LOAD 'stores' AS (store_id:chararray, name:chararray);
sales = LOAD 'sales' AS (store_id:chararray, price:int);
nonnull_stores = FILTER stores BY store_id IS NOT NULL;
nonnull_sales = FILTER sales BY store_id IS NOT NULL;
joined = JOIN nonnull_stores BY store_id, nonnull_sales BY store_id;
```


Summary

You can boost performance by eliminating unneeded data during processing.

Pig's error messages don't always clearly identify the source of a problem.

- **It is recommend that testing your scripts with a small data sample.**
- **Looking at the Web UI, and especially the log messages, can be helpful.**



Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
 - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed., Sebastopol, California: O'Reilly, 2015.
 - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
 - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017



Unit Summary

- MapReduce Code Explanation.

Summary

